

BCA182

FREE RTOS MULTISENSOR SYSTEM

Laboratory Report

Submitted by: Renato G. Beroy

Section: BCA182 B187

Instructor: Prof. Paul Rodolf P. Castor, M.Sc.

Date: 28 September 2026

Abstract

This laboratory developed and verified a FreeRTOS-based multisensor application for an STM32F103 platform. The work focused on modularizing task responsibilities, preserving inter-task communication through FreeRTOS queues and event groups, and validating hardware-independent logic through automated unit tests and static analysis. The final firmware build completed successfully. A Unity-based native test harness executed 15 tests with 0 failures, while Cppcheck completed successfully with 0 high-severity, 0 medium-severity, and 35 low-severity findings. The low-severity findings were primarily unused-function and style observations produced while examining embedded modules outside their runtime call graph.

1. Introduction

Embedded real-time applications benefit from clear separation of responsibilities because sensing, user input, alarm handling, display updates, and system-state management operate concurrently. In this laboratory, the original FreeRTOS multisensor application was reorganized into dedicated source modules so that each task had a focused responsibility and shared system resources remained explicit. The verification phase then exercised the logic that could be tested independently of physical hardware.

The target application uses an STM32F103 microcontroller and FreeRTOS. The application includes a DHT22 temperature/humidity sensor, light sensing, a PIR motion input, an OLED display, a rotary encoder, and a buzzer. FreeRTOS queues carry sensor data, while an event group communicates motion and active/inactive state information between tasks.

2. Objectives

- Separate the main FreeRTOS responsibilities into maintainable source modules.
- Preserve the required sensor, display, alarm, motion, input, and state behavior after modularization.
- Create and execute a minimum of 13 meaningful unit tests for hardware-independent logic.
- Run static analysis using Cppcheck and document the resulting severity levels.
- Produce verifiable build and test evidence suitable for laboratory submission.

3. System Overview

At startup, `main.cpp` initializes the hardware interfaces, creates the sensor queue, alarm queue, and system event group, and creates the FreeRTOS tasks. The application then starts the FreeRTOS scheduler. The task structure and supporting logic were separated so that changes to one functional area do not require placing all behavior in a single task source file.

FreeRTOS Multisensor Modular Architecture

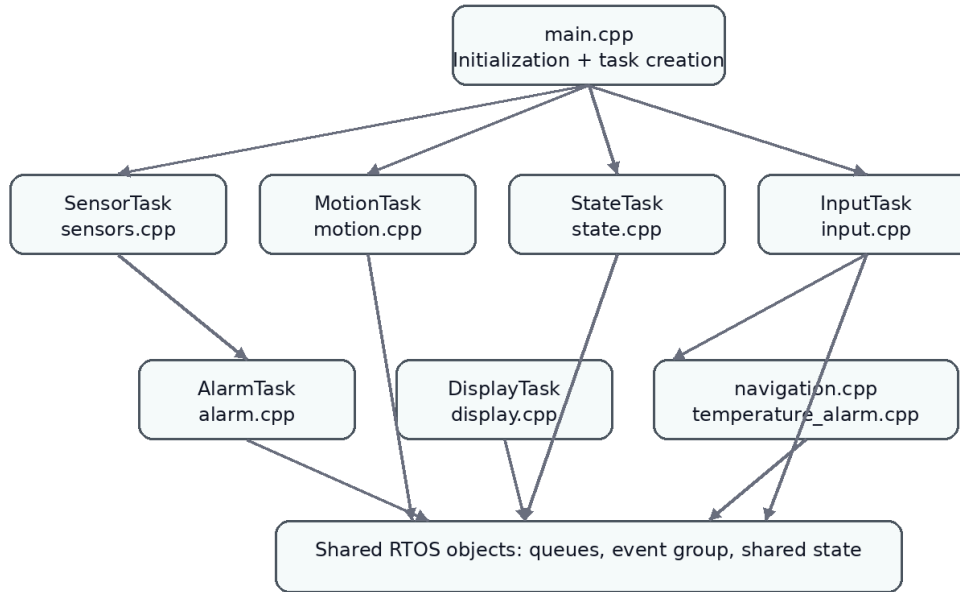


Figure 1. Final modular architecture of the FreeRTOS multisensor application.

Module	Primary responsibility	Verification focus
sensors.cpp	Sensor acquisition and sensor queue publishing	Sensor data structure and runtime integration
input.cpp	Rotary encoder polling and display navigation	Navigation transitions
motion.cpp	PIR monitoring and motion event generation	Motion event signaling
state.cpp	ACTIVE/INACTIVE state management and inactivity timeout	State transition contract
alarm.cpp	Temperature alarm task and buzzer control	Alarm threshold logic
display.cpp	OLED state and screen rendering	Display-mode behavior
navigation.cpp	Pure display-mode transition logic	Clockwise/counterclockwise navigation tests
temperature_alarm.cpp	Pure temperature threshold logic	Five threshold tests

4. Implementation and Modularization

4.1 Task separation

The task implementation was divided so that each FreeRTOS task resides in its own functional module. SensorTask remains responsible for collecting sensor information. InputTask handles rotary encoder movement and maps clockwise or counterclockwise steps to the display sequence. MotionTask polls the PIR input and sets the EVENT_MOTION bit when motion is detected. StateTask maintains ACTIVE/INACTIVE behavior using a 15-second inactivity threshold. AlarmTask evaluates temperature conditions and controls the buzzer based on both alarm state and system state. DisplayTask controls OLED power and updates the selected display mode.

4.2 Shared communication

Two FreeRTOS queues are used for sensor data: one for the display path and one for the alarm path. A FreeRTOS event group carries EVENT_MOTION and EVENT_ACTIVE state information. Shared handles and system state are declared in the application interface so that the task modules can communicate without duplicating global definitions.

4.3 Temperature alarm behavior

The temperature classification is based on the required thresholds. A temperature below 18.0 degrees C produces LOW_TEMPERATURE, a temperature from 18.0 through 30.0 degrees C is NORMAL, and a temperature above 30.0 degrees C produces HIGH_TEMPERATURE. Buzzer activation additionally requires the overall system to be ACTIVE.

5. Unit Testing

Unit testing was performed using Unity with a native Windows test harness. Hardware-dependent FreeRTOS and STM32 components were excluded from the native execution path so that hardware-independent logic could be compiled and exercised directly. The final test program executed 15 tests.

5.1 Test coverage

Category	Number of tests	Coverage
Temperature alarm	5	Below 18, exactly 18, normal mid-range, exactly 30, above 30
Display navigation	6	Clockwise progression, reverse progression, wrap-around behavior
System state/event contract	4	ACTIVE vs INACTIVE, 15-second timeout, motion and active event bits
Total	15	All required categories covered

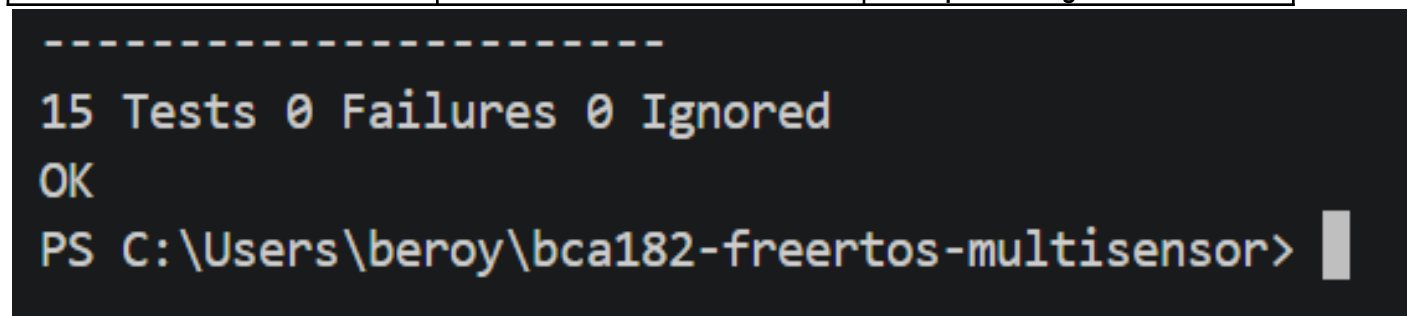


Figure 2. Unity console record showing 15 executed tests, 0 failures, and an overall OK result.

The final unit-test result was: 15 Tests, 0 Failures, 0 Ignored, OK. This exceeds the minimum requirement of 13 meaningful tests. The tests focus on deterministic logic rather than hardware timing or peripheral behavior, which makes them appropriate for a native host-side test harness.

6. Build Verification

After modularization, the STM32 project was rebuilt with PlatformIO. The final embedded firmware build completed successfully. This confirms that the separated task modules, headers, queue/event interfaces, and remaining application components linked into a valid firmware image.

```
Firmware build evidence

PS C:\Users\beroy\bca182-freertos-multisensor>
platformio.exe run -e bluepill_f103c8
...
===== [SUCCESS] =====
```

Figure 3. PlatformIO firmware build evidence showing a successful build.

7. Static Analysis

Cppcheck was executed through PlatformIO using the native environment. The analysis completed with a PASSED status. The report contained no high- or medium-severity findings. There were 35 low-severity findings, dominated by unused-function/style observations such as task entry points and hardware functions that are called through the embedded runtime rather than from the host-side analysis entry point.

```
Problems 3 Output Debug Console Terminal Ports MEMORY xRTOS

Total      0      0      35

Environment Tool      Status      Duration
-----
native      cppcheck  PASSED      00:25:48.189
===== 1 succeeded in 00:25:48.189 =====
PS C:\Users\beroy\bca182-freertos-multisensor>
```

Figure 4. Cppcheck static-analysis record showing 0 high, 0 medium, and 35 low findings.

Severity	Count	Interpretation
High	0	No high-severity findings reported
Medium	0	No medium-severity findings reported
Low	35	Primarily style/unused-function observations
Overall	PASSED	Static-analysis run completed successfully

8. Discussion

The modularization improved separation of concerns. Before the refactoring, several task implementations were grouped in a single task source file. After the refactoring, task responsibilities were distributed across focused modules, making the structure easier to inspect and maintain. The successful firmware build provides the primary evidence that the refactoring preserved the required link relationships.

The test phase also exposed an important distinction between embedded integration testing and native logic testing. The embedded PlatformIO test flow expected a target connection for execution, while the selected logic tests could be compiled and executed directly on the host. The resulting Unity executable ran all 15 tests successfully. This approach validated deterministic logic without requiring the STM32 hardware to be connected to the test process.

The static analysis stage reported 35 low-severity observations. Most are expected in an embedded project because functions such as task entry points, peripheral initialization routines, and hardware access functions are not naturally reachable from a host-side analysis entry point. The absence of high- and medium-severity findings indicates that the analysis did not identify more serious classes of issues in the examined source.

9. Results Summary

Verification activity	Result	Evidence
STM32 firmware build	SUCCESS	Figure 3 / PlatformIO console record
Unity native unit tests	15 passed, 0 failed	Figure 2 / Unity console record
Static analysis	PASSED; 0 high, 0 medium, 35 low	Figure 4 / Cppcheck record

10. Conclusion

The BCA182 FreeRTOS multisensor laboratory was successfully completed through modularization, unit testing, and static analysis. The FreeRTOS task responsibilities were separated into dedicated modules for sensing, input, motion, state, alarm, and display control, with supporting navigation and temperature-alarm logic isolated into reusable components. The final embedded build completed successfully.

The verification stage produced 15 passing Unity tests with zero failures, exceeding the required minimum of 13 meaningful tests. The tests covered temperature alarm thresholds, display navigation, system-state values, inactivity timing, and event-bit definitions. Cppcheck also completed successfully with zero high-severity and zero medium-severity findings. These results provide evidence that the modularized application is structurally sound and that its testable logic behaves according to the defined requirements.

Appendix A. Test Cases

#	Test case	Expected result
1	Temperature 17.9 degrees C	LOW_TEMPERATURE
2	Temperature 18.0 degrees C	NORMAL
3	Temperature 25.0 degrees C	NORMAL
4	Temperature 30.0 degrees C	NORMAL
5	Temperature 30.1 degrees C	HIGH_TEMPERATURE
6	TEMPERATURE clockwise	HUMIDITY
7	HUMIDITY clockwise	MOTION
8	TEMPERATURE counterclockwise	ALERT
9	ALERT counterclockwise	LIGHT
10	ALERT clockwise	TEMPERATURE
11	ACTIVE and INACTIVE enum values	Distinct states
12	15-second inactivity threshold	15000 ms

#	Test case	Expected result
13	EVENT_MOTION definition	Nonzero / bit 0
14	EVENT_ACTIVE definition	Nonzero / bit 1
15	Additional navigation wrap-around check	Expected reverse transition

Appendix B. Verification Notes

The native unit-test environment initially attempted to use the embedded target flow, which required an upload/test port and OpenOCD connection. To isolate hardware-independent logic, the final verification used GCC and Unity directly on Windows. The resulting native executable returned an overall OK result with 15 tests and 0 failures. The application firmware was separately validated by a successful STM32 PlatformIO build.

A package-manifest warning appeared during PlatformIO tooling activity. It did not prevent the successful firmware build, direct Unity execution, or Cppcheck pass documented in this report.